

Secure Railway Reservation System

Project Specification & Security Requirements

Student Documentation

This document defines the scope, functional requirements, architecture, trust boundaries, security requirements, and API/interface specifications for the Secure Railway Reservation System (SRRS).

Programming-language implementation requirement

The project shall be implemented using **Python, Java, C++, and Rust**. The same functional requirements, architecture, security requirements, and interface specifications shall be maintained across all four implementations.

1. Scope

The **Secure Railway Reservation System (SRRS)** is a deliberately small software system designed to support railway reservation activities while providing a common basis for studying software security across different programming languages.

The system will be implemented using **Python, Java, C++, and Rust** while maintaining the same functional requirements, architecture, security requirements, and interface specifications.

The project focuses on the implementation of security requirements across programming languages, with particular emphasis on **software interactions, trust boundaries, secure implementation, and security properties**.

1.1 In Scope

- Passenger authentication
- Train search
- Seat availability
- Seat reservation
- Viewing reservations
- User identity and authorization
- Input and data validation
- Reservation integrity
- Concurrent booking
- Secure interaction between software components

1.2 Out of Scope

- Payment processing
- Ticket pricing
- Cancellation and refund
- Railway scheduling
- Train operation and control
- Real-world railway system integration
- Notification services
- Administrative management functions

The same functional and security requirements shall be maintained across the four programming-language implementations.

2. Functional Requirements, Users and Operations

2.1 Users

Passenger

An authenticated passenger can authenticate with the system, search for trains, view available seats, book an available seat, and view their own reservations.

The system also contains internal software components that perform operations on behalf of the passenger.

2.2 Functional Requirements

ID	Functional Requirement
FR-01	The system shall authenticate a passenger before allowing protected operations.
FR-02	The system shall allow a passenger to search for available trains.
FR-03	The system shall allow a passenger to view available seats for a selected train.

ID	Functional Requirement
FR-04	The system shall allow an authenticated passenger to book an available seat.
FR-05	The system shall allow a passenger to view their reservations.
FR-06	The system shall maintain reservation and seat information consistently.

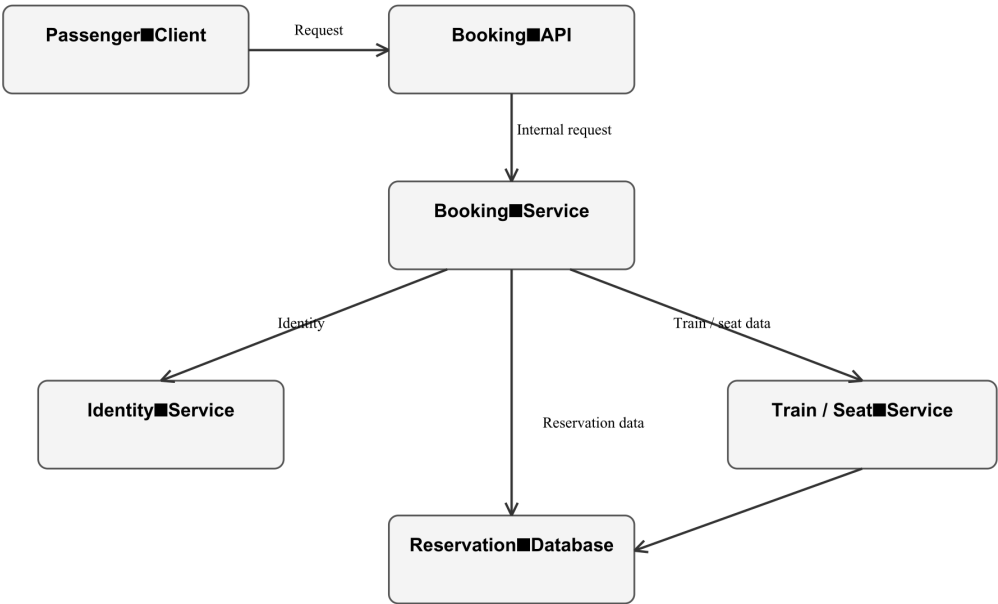
2.3 Core Operations

Authenticate → Search Train → View Available Seats → Book Seat → View Reservation

These operations form the functional basis of the system and its security requirements.

3. System Architecture

The system consists of the following logical components:



3.1 Components and Responsibilities

Component	Responsibility
Passenger Client	Collects user requests and displays system results.
Booking API	Receives passenger requests and provides the external interface to the system.
Booking Service	Implements booking-related business logic.
Identity Service	Authenticates users and provides authenticated identity information.
Train/Seat Service	Provides train and seat information and manages seat availability.
Reservation Database	Stores reservation and related system data.

3.2 Data Flow

Passenger → Booking API → Booking Service → Identity Service / Train/Seat Service → Reservation Database → Booking Result → Passenger

A typical booking operation follows this data flow. The implementation shall preserve the specified security requirements throughout these data flows.

3.3 Interaction Model

ID	Interaction	Description
I1	Passenger → Booking API	Passenger submits a system request.
I2	Booking API → Booking Service	API forwards an operation for processing.
I3	Booking Service → Identity Service	Booking Service obtains or validates user identity information.
I4	Booking Service → Train/Seat Service	Booking Service obtains and updates seat information.
I5	Booking Service → Reservation Database	Reservation information is stored or retrieved.

4. Trust Boundaries

A **trust boundary** represents a point where data, requests, identity, or control moves between entities with different levels of trust.

ID	Trust Boundary	Source	Destination	Information Crossing Boundary
TB-01	External User Boundary	Passenger	Booking API	Credentials, requests, booking parameters
TB-02	Application Component Boundary	Booking API	Booking Service	User identity, operation, request parameters
TB-03	Service Boundary	Booking Service	Identity Service	Authentication and identity information
TB-04	Service Boundary	Booking Service	Train/Seat Service	Train ID, seat information, availability/update requests
TB-05	Data Boundary	Booking Service	Reservation Database	Reservation and user data

5. Security Requirements

The following security requirements apply to all programming-language implementations of the system.

ID	Security Requirement	Security Property
SR-01	Only authenticated users shall be permitted to perform protected booking operations.	Authentication
SR-02	A passenger shall be permitted to create a reservation only for themselves.	Authorization / Identity Integrity
SR-03	A passenger shall be permitted to view only their own reservations.	Authorization / Confidentiality
SR-04	Externally supplied input shall be validated before being used by the application or underlying components.	Input Integrity
SR-05	A seat shall not be successfully allocated to more than one passenger.	Integrity / Concurrency
SR-06	Reservation data shall not be modified through unauthorized operations.	Integrity
SR-07	Memory operations shall not permit access outside the intended memory object or buffer where the programming language permits such operations.	Memory Safety

Security Requirement Scope

The security requirements deliberately include properties that may be influenced differently by the programming language, runtime environment, framework, architecture, and programmer implementation.

The requirements shall remain constant across the language implementations.

6. API / Interface Specification

The API specification defines the required operations, inputs, outputs, and security expectations. It does not prescribe a particular programming language or implementation mechanism.

ID	Operation	Interface	Input	Output	Security Requirement
API-01	Authenticate	authenticate(credentials)	Username, Password	Authentication result; Authenticated Identity	Authenticated identity shall be established before protected operations.
API-02	Search Train	search_train(criteria)	Train ID / Search Criteria	Matching train information	Input shall be appropriately validated.
API-03	View Available Seats	get_available_seats(train_id)	Train ID	List of available seats	Train identifier and returned information shall be handled securely.
API-04	Book Seat	book_seat(authenticated_user, train_id, seat_no)	Authenticated User, Train ID, Seat Number	Booking Result, Reservation Information	Authentication, authorization, input validation, seat availability and reservation integrity shall be enforced.
API-05	View Reservation	get_reservation(authenticated_user, reservation_id)	Authenticated User, Reservation ID	Reservation Information	A passenger shall not be permitted to retrieve another passenger's reservation.

End of Project Specification

Student Starter Package

The **Student Starter Package** will be provided separately. It will contain the programming-language-specific starter code, project environment, test framework, and implementation materials required to develop and evaluate the system.

The starter package will **not provide the completed secure implementation**. Students will be required to implement the specified functionality and security requirements as part of the assignment.